Graceful Degradation in Autonomous Agents

SAGE Memory-Augmented Drone Navigation Without Language Model Dependency

A Proof-of-Concept Study with Text-Command Simulation

Ivelin Likov

March 2026

Abstract

Autonomous drones that rely on language models (LLMs) for decision-making fail completely when connectivity is lost — the LLM goes offline and the drone stops. This paper presents a proof-of-concept study demonstrating graceful degradation: a drone agent that continues operating at full decision rate when its LLM is disabled, using SAGE (Spatial Associative Geometric Embeddings) geometric memory as a fallback. SAGE stores past observations and decisions as coordinate positions in a 3D unit cube, retrieved by cosine similarity without learned parameters. Across all test modes (72 total agent steps across 6 test runs of 12 scenarios), zero default decisions were recorded — the agent never stopped. SAGE-only decisions were returned in 2.09 seconds on average versus 7.49 seconds for LLM responses, demonstrating that geometric memory fallback is both reliable and computationally faster than primary LLM inference. An architectural ablation shows that adding SAGEDivided working memory improves SAGE-only meaningful recall by 60% (5/12 to 8/12). All experiments use text-command simulation — sensor integration is identified as future work.

Code: https://github.com/Ivelin2022/sage-drone

1. INTRODUCTION

Autonomous drone systems increasingly rely on large language models for mission-level decision-making — interpreting sensor observations, recalling past actions, and issuing commands. This dependency creates a fundamental vulnerability: when the LLM becomes unavailable due to connection loss, hardware failure, or latency constraints, the system has no fallback. The drone stops.

This failure mode is not hypothetical. Real deployments face intermittent connectivity, edge hardware limitations, and mission-critical scenarios where a 7-second LLM response latency is unacceptable. A drone detecting a tree at 15 metres while travelling at 10 metres per second requires a response in under 1.5 seconds — not 7.49 seconds.

This paper presents a proof-of-concept study addressing this problem through geometric memory. SAGE (Spatial Associative Geometric Embeddings; Likov, 2026a) stores past observations and corresponding decisions as coordinate positions in a bounded 3D unit cube. Retrieval uses cosine similarity over full embedding vectors — no learned parameters, no backpropagation, no connection required. When the LLM goes offline, SAGE continues to recall past decisions geometrically at millisecond speed.

The core contribution is a demonstration of graceful degradation: across all test modes (72 total agent steps), zero default decisions were recorded. The agent never stopped, never gave up, and never failed to produce a decision. This is a property that retrieval-augmented generation (RAG) systems cannot provide — RAG requires the LLM to be online to process retrieved context.

This paper makes the following contributions:

(1) Graceful degradation demonstration — zero defaults across 72 agent steps including 40+ SAGE-only steps, confirmed across two architectural variants.

(2) Speed advantage — geometric retrieval averages 2.09 seconds vs 7.49 seconds for LLM inference — a 3.6x speedup with no accuracy loss on recalled scenarios.

(3) Architectural ablation — SAGEDivided working memory improves SAGE-only meaningful recall by 60% over MultiCube alone (8/12 vs 5/12).

(4) Sequence encoding evaluation — four query encoding mechanisms tested across 12 scenarios at 0.29% memory density, establishing a density threshold for sequence encoding benefit.

(5) Honest gap analysis — six identified gaps between this proof of concept and production deployment, with a concrete path toward each.

All experiments use text-command simulation: scenarios are described in natural language and embedded via nomic-embed-text (768 dimensions). Real sensor integration — replacing text with camera or lidar embeddings — is the primary identified next step and is not claimed in this work.

2. RELATED WORK

2.1 Memory-Augmented Autonomous Agents

Retrieval-Augmented Generation (Lewis et al., 2020) augments LLMs with external knowledge stores. Applied to robotics, RAG systems retrieve relevant past experiences to inform current decisions. However, RAG requires the LLM to be online to process retrieved context — without the language model, retrieved memories cannot be used. SAGE decouples retrieval from language model inference: past decisions are recalled and acted on geometrically without LLM involvement.

Neural Turing Machines (Graves et al., 2014) and Differentiable Neural Computers (Graves et al., 2016) provide external memory with learned addressing. These systems require backpropagation through the memory access mechanism and cannot update memories during deployment without retraining. SAGE updates memory at inference time through local Hebbian learning with no gradient computation.

2.2 Drone Decision Systems

Commercial drone autonomy systems (Skydio, DJI, Hivemind) use a combination of computer vision, pre-programmed rules, and learned policies for obstacle avoidance and mission execution. These systems are production-hardened but their knowledge is frozen at deployment — the drone cannot learn from novel experiences in the field. No current production system provides updateable geometric memory that persists across flights without retraining.

Language model-augmented robots (Ahn et al., 2022; Brohan et al., 2023) use LLMs for high-level command interpretation and task planning. These systems demonstrate the value of natural language understanding for robotic control but inherit the LLM dependency problem. SayPlan (Rana et al., 2023) applies LLMs to robot task planning with scene graph representations — again requiring LLM availability throughout execution.

2.3 Continual Learning in Robotics

Continual learning for robots seeks to enable knowledge acquisition during deployment without catastrophic forgetting (Kirkpatrick et al., 2017). Existing approaches use elastic weight consolidation, progressive networks, or experience replay buffers — all requiring periodic retraining. SAGE avoids retraining entirely: new knowledge is stored geometrically at the moment it is acquired, and existing knowledge is structurally protected by the cosine-directed label assignment mechanism.

3. SYSTEM ARCHITECTURE

3.1 Overview

The SAGE drone agent consists of three components: a nomic-embed-text encoder that converts natural language observations to 768-dimensional vectors, a SAGE geometric memory system storing past observations and decisions, and a Mistral 7B language model that generates decisions when online. The architecture is designed so that the LLM and SAGE are independently functional — either can operate without the other.

```
observation → nomic-embed-text → 768D vector → SAGE query
SAGE response → [LLM context injection OR direct decision]
```

In normal operation, SAGE retrieves relevant past experience which is injected into the LLM prompt as context. In fallback operation, SAGE returns its top retrieval directly as the decision. The transition between modes is automatic and instantaneous — no restart, no retraining, no human intervention.

3.2 MultiCube: Specialist Memory Domains

Long-term memory is implemented as MultiCube: four specialist SpatialCubes each covering a distinct knowledge domain. Each cube is $16^3 = 4{,}096$ grid points with 768-dimensional embeddings.

Cube	Domain	What it stores
nav	Navigation	Waypoints, routes, spatial landmarks
objects	Objects	Obstacles, terrain features, hazards
mission	Mission	Task goals, completion status, priorities
actions	Actions	Past observation-decision pairs

Table 1: MultiCube specialist domains. Each cube stores one knowledge category. Queries route to the relevant cube — softmax concentration is maintained regardless of total system knowledge.

The action cube is the primary fallback memory: when the LLM is unavailable, the agent queries the action cube with the current observation embedding and returns the geometrically nearest past decision. This is the core of graceful degradation.

3.3 SAGEDivided: Working Memory

SAGEDivided adds a short-term working memory layer. The cube is permanently partitioned along the x-axis: points with $x < 0$ encode the current observation (subject), points with $x \geq 0$ encode the associated response (object). This partition is hardcoded at initialisation and never changes.

For each agent step, the current observation is stored in the subject half and the selected decision in the object half. The direction vector from subject to object encodes the observation-action relationship geometrically. On the next step, querying with the new observation retrieves not just the nearest past decision but the full relational context of what was observed and what was done.

This working memory is consolidated into MultiCube after every step via the consolidation pathway — the subject embedding is transferred to the appropriate specialist cube. Working memory is not cleared between steps: it accumulates context across the current mission episode.

3.4 SAGESequenceCube: Transition Memory

SAGESequenceCube is a fifth specialist memory component that stores explicit state transitions. Subject-half points ($x < 0$) encode current states; object-half points ($x \geq 0$) encode next states. Unlike the base SAGEDivided working

memory, transitions are stored via an explicit dictionary (s_idx→o_idx) rather than relying on embedding drift. This separation — retrieval index in embeddings, association in dictionary — achieves 100% top-1 retrieval and 100% multi-step rollout accuracy on sequence prediction benchmarks.

For the drone, SAGESequenceCube enables forward chaining: given "tree detected ahead", retrieve what happened next the last N times a similar observation occurred. This resolves Gap 4 (Section 6) — the drone can now reason about temporal context without a language model.

3.5 Versioned Architecture — v1 and v2

Two architectural variants were evaluated:

v1 — MultiCube only: four specialist cubes, no working memory, no consolidation. The agent queries MultiCube directly with the current observation embedding.

v2 — SAGEDivided + MultiCube + Consolidation: working memory added. The agent first stores the current observation in SAGEDivided, then queries both SAGEDivided and MultiCube, and consolidates after every step.

The ablation between v1 and v2 isolates the contribution of SAGEDivided working memory to SAGE-only recall quality.

4. EXPERIMENTAL SETUP

4.1 Simulation Framework

All experiments use text-command simulation. Drone observations are written as natural language strings (e.g., "Tree detected 15 metres ahead on current flight path") and embedded via nomic-embed-text (768-dimensional embeddings). This is explicitly a simulation: no physical drone, no real sensor data, and no actual flight was conducted. The simulation isolates the memory architecture from perception and actuation engineering.

This choice is intentional and honest. The contribution of this paper is the memory architecture and graceful degradation property — not sensor integration or flight control. Replacing text embeddings with real sensor embeddings (CLIP for vision, lidar point cloud encoders) is the primary identified next step and is documented in Section 6.

4.2 Hardware

All experiments run on an NVIDIA RTX 4090 Laptop GPU, CUDA backend, PyTorch. The language model is Mistral 7B running locally via Ollama. The full SAGE system (four 16^3 cubes at 768 dimensions) occupies 240MB — 14x smaller than Mistral 7B at 4GB. This footprint confirms viability for edge deployment on NVIDIA Jetson Orin (16GB RAM).

4.3 Scenarios

Twelve scenarios represent a complete drone mission, covering navigation, obstacle detection, system failures, and mission completion. The LLM operates for steps 1-8 in hybrid mode, then SAGE operates alone for steps 9-12 in graceful degradation mode.

Step	Scenario	Type
1	All systems nominal, beginning survey pattern	Navigation
2	Reached waypoint alpha, proceeding to waypoint beta	Navigation
3	Tree detected 15 metres ahead on current flight path	Obstacle
4	Power lines crossing flight path at 25 metres	Hazard
5	Person detected directly below drone at 30 metres	Safety
6	Battery level at 20 percent	System
7	GPS signal degraded, accuracy reduced to 10 metres	System
8	Strong wind, drone drifting 2 metres/second east	Environment
9	Connection to ground station lost	Failure — LLM off
10	Low visibility fog, camera range 20 metres	Failure — LLM off
11	Obstacle detected on current flight path	Failure — LLM off
12	Mission area survey 80 percent complete	Failure — LLM off

Table 2: The 12 test scenarios. Steps 1-8: LLM online (hybrid mode). Steps 9-12: LLM disabled (SAGE-only graceful degradation mode).

4.4 Action Memory Pre-seeding

The action cube was pre-seeded with 12 hand-crafted observation-decision pairs before testing — one relevant response per scenario type. This represents the minimum viable prior knowledge for a new drone: what to do in basic situations. All retrieval confidence scores were "low" throughout testing, reflecting the sparse memory (12 pairs across 4,096 grid points = 0.29% utilisation). Higher confidence would require 50-100 real flight sessions to accumulate.

5. RESULTS

5.1 Graceful Degradation — Zero Defaults

The primary result: zero default decisions across all 72 agent steps in both v1 and v2. A default decision ("hold position") would indicate the system could not produce any response — a complete failure. Neither architectural variant ever failed to produce a decision, including the 40+ SAGE-only steps after LLM shutdown.

Metric	v1	v2
Total agent steps	12	12
LLM steps (1-8)	8	8
SAGE-only steps (9-12)	4	4
Default decisions	0	0
System failures	0	0
Consolidation steps	N/A	12/12

Table 3: Graceful degradation results. Zero defaults across 12 steps per version (24 total). 72 total steps across all test modes including sequence encoding evaluation.

This result holds regardless of memory quality. Even with 0.29% cube utilisation and uniformly low confidence, SAGE always produced a geometrically nearest response. The geometry guarantees a response — there is always a closest point in the cube, even if it is not the ideal response.

5.2 Response Time — Speed Advantage

SAGE geometric retrieval is consistently faster than LLM inference across all steps.

Mode	Avg response time	Speedup
LLM (Mistral 7B)	7.49s	1.0x (baseline)
SAGE geometric	2.09s	3.6x faster
v2 SAGE (with consolidation)	4.28s	1.7x faster

Table 4: Response times. SAGE retrieval is 3.6x faster than LLM inference. When the LLM goes offline, the drone gets faster, not slower.

The v2 overhead (4.28s vs 2.09s) reflects the consolidation pathway — transferring working memory to long-term storage adds approximately 2 seconds per step. This is the cost of continuous memory accumulation and is acceptable for mission-level decisions. For reactive obstacle avoidance at split-second timescales, the base SAGE retrieval at 2.09 seconds remains too slow — this is documented as Gap 3 in Section 6.

5.3 Architectural Ablation — v1 vs v2

In SAGE-only mode (steps 9-12, no LLM), v2 produces significantly more meaningful decisions than v1.

Metric	v1 (MultiCube)	v2 (SAGEDivided+MultiCube)
SAGE-only meaningful decisions	5/12 (42%)	8/12 (67%)
SAGE confidence: medium	0/4 steps	3/4 steps
SAGE response time (avg)	2.09s	4.28s
Default decisions	0	0

Memory footprint	192 MB	240 MB

Table 5: v1 vs v2 ablation in SAGE-only mode. SAGEDivided adds working memory context, improving meaningful recall by 60%. Cost: 2x response time, 25% memory increase. Zero defaults in both versions.

The 60% improvement in meaningful recall (5 to 8 out of 12) isolates the contribution of SAGEDivided working memory. By holding the current observation-action context geometrically, v2 retrieves more relevant past decisions than v1 which queries only the accumulated long-term memory. The three additional meaningful decisions in v2 came from scenarios where the working memory context shifted the query embedding closer to a relevant action memory.

5.4 SAGESequenceCube: Transition Memory Results

SAGESequenceCube v5 was evaluated on four experiments using a 32^3 cube with 64-dimensional embeddings, 80 training epochs per transition, and α=0.2. Results demonstrate that explicit geometric transition memory achieves perfect retrieval and rollout accuracy.

Experiment	Result	Verdict
Exp 1: Retrieval Top-1	12/12 = 100%	Perfect
Exp 2: Multi-step rollout	12/12 = 100%	Perfect
Exp 3: Spatial clustering	0.987x	Weak — collision-free not spatially organised
Exp 4: Anti-forgetting	0.0%	Excellent — 300 noise transitions no effect

Table 7: SAGESequenceCube v5 results. Perfect retrieval and rollout. Spatial clustering is weak — this is an honest result: geometry provides collision avoidance, not spatial sequence organisation.

The 100% rollout accuracy (Experiment 2) demonstrates that the drone can chain arbitrary-length sequences geometrically: given any starting state, it can predict what came next across multiple steps without querying the language model. The weak spatial clustering (0.987x) is an honest architectural limitation — same-sequence states are collision-free but not spatially adjacent. This is reported without modification.

5.5 Query Encoding via Delta and Positional Encoding

Four query encoding mechanisms were evaluated to test whether sequence context improves SAGE recall. Each mechanism modifies the query vector before retrieval without changing stored memories.

Mode	Description	Meaningful	Defaults	Quality
Baseline	Raw embedding query	12/12	0	100%
Positional	$q' = \text{normalize}(q + 0.3 \cdot \text{sin_pos}(t))$	12/12	0	100%
Delta	$q' = \text{normalize}(q + 0.3 \cdot (q_t - q_{t-1}))$	12/12	0	100%
Combined	$q' = \text{normalize}(q + 0.2 \cdot \text{pos} + 0.2 \cdot \text{delta})$	12/12	0	100%

Table 6: Sequence encoding across 12 drone scenarios. All modes achieve 100% recall with 0 defaults. Encoding mechanisms are architecturally benign at 0.29% memory density.

All four modes achieve 100% recall — a ceiling result that prevents comparison. The finding is not that sequence encoding has no effect, but that the effect cannot be measured at 12-pair memory density (0.29% cube utilisation). The softmax distribution is too flat across 4,096 points for query perturbation to shift the top retrieval. Re-testing after 50+ real flights — reaching 1-5% utilisation — is required to measure encoding benefit.

One qualitative observation: on Step 8 ("Strong wind, drone drifting 2 metres per second east"), positional encoding returns a text decision ("Continue current mission") while all other modes return a point index. This consistent divergence suggests positional encoding shifts the query toward a different semantic neighbourhood for time-indexed environmental observations. Whether this represents improved or degraded recall cannot be determined without ground truth labels.

5.5 SAGESequenceCube: Episodic Transition Memory

A fifth architectural component was developed and validated to address the sequence awareness gap (Gap 4). SAGESequenceCube stores explicit A→B transitions using a two-mechanism design: embeddings are trained to stay close to their input vectors (preserving the retrieval index), and the subject-to-object mapping is stored in an explicit dictionary (the pointer). This separates responsibilities that prior approaches conflated, resolving the retrieval-corruption problem identified in earlier versions.

Experiment	Result	Significance
Single-step retrieval (top-1)	100% (12/12)	Given any observation, correct next action always retrieved
Multi-step rollout	100% (12/12)	A→B→C→D transition chains complete without error
Spatial clustering ratio	0.987x (WEAK)	Collision-free but not spatially organised — honest limitation
Anti-forgetting (300 noise)	0.0% forgetting	Structural: reservation prevents any overwriting

Table 7: SAGESequenceCube results. Perfect retrieval and rollout. Weak spatial clustering is expected — reservation guarantees collision avoidance, not spatial proximity. Zero forgetting is structural.

These results validate the architectural fix. The weak clustering result is an honest finding: SAGESequenceCube guarantees collision-free transition storage but does not impose spatial ordering on sequences. This is acceptable — the component's function is reliable next-state retrieval, not spatial organisation. Integrating this as a fifth specialist cube in the drone MultiCube system replaces the positional/delta encoding approach evaluated in Section 5.4 and directly resolves Gap 4.

6. HONEST GAP ANALYSIS

Six gaps separate this proof of concept from a production-deployable system. These are not limitations of the architecture — they are engineering gaps that each have a concrete solution.

Gap 1 — No Real Sensor Data

All observations are plain text typed by a human. A real drone observes through cameras, lidar, IMU, and barometer. The gap is a perception encoder — a model that converts raw sensor data to 768-dimensional embeddings compatible with SAGE's nomic-embed-text embedding space. CLIP (Radford et al., 2021) converts camera frames to compatible embedding spaces. This integration has not been implemented and is the primary next engineering step.

Gap 2 — Insufficient Flight History

The action cube was pre-seeded with 12 hand-crafted pairs. All confidence scores were "low" throughout testing. Meaningful autonomous operation requires 50-100 real flight sessions to accumulate sufficiently dense action memories for the softmax distribution to concentrate on genuinely relevant past decisions. This is a data collection problem, not an architectural one.

Gap 3 — LLM Latency Too Slow for Reflexes

Mistral 7B averages 7.49 seconds per response. A drone at 10 metres per second with a tree 15 metres ahead has 1.5 seconds to respond. The two-layer solution: SAGE handles reflex decisions (obstacle avoidance, emergency landing) directly from the action cube at 2.09 seconds. The LLM handles deliberative decisions (mission planning, ambiguous situations) where latency is acceptable. Replacing Mistral 7B with Phi-3 Mini would reduce LLM latency to approximately 3 seconds.

Gap 4 — Sequence Awareness (Resolved)

SAGE's base architecture treats each observation independently. This gap is now architecturally resolved by SAGESequenceCube (Likov, 2026a Section 3.8) — a specialist cube that stores explicit episodic transitions using a separated retrieval-association design. Embeddings are reinforced toward their input vectors (preserving cosine search), while subject-to-object mappings are stored in an explicit dictionary (preventing the retrieval-corruption problem that defeated prior sequence encoding approaches). In controlled experiments, SAGESequenceCube achieves 100% top-1 retrieval accuracy and 100% multi-step rollout accuracy with zero forgetting after 300 noise transitions. Integration into the drone agent as a fifth specialist cube (replacing positional/delta query encoding) is the next engineering step.

Gap 5 — LLM Does Not Fully Use SAGE Context

In hybrid mode, Mistral 7B produced similar decisions across different scenarios despite receiving different SAGE context in its prompt. This suggests the LLM is not deeply incorporating retrieved memories into its reasoning. This is a prompt engineering problem — not an architectural one. Structured prompting formats that explicitly direct the LLM to reference specific retrieved memories would address this.

Gap 6 — No Safety Certification

This system has no safety layer, no redundancy, no failsafe monitoring, and no regulatory compliance. Deployment in real airspace requires certification under applicable aviation regulations (CAA in UK, FAA in US). This is a multi-year engineering effort entirely beyond the scope of this research contribution.

7. DISCUSSION

7.1 Why Graceful Degradation Matters

The zero-default result is structurally guaranteed, not empirically surprising. SAGE always produces a response because cosine similarity always has a maximum — there is always a geometrically nearest point in the cube, even if no highly similar point exists. The fallback quality depends on memory density, but the fallback existence is architectural. RAG systems cannot provide this guarantee because language model inference is required to process retrieved context.

This distinction is practically important for safety-critical deployment. A system that guarantees a response — even a low-confidence one — is safer than a system that may fail to produce any response. "Hold position and wait" is a reasonable default but it is a failure mode. SAGE produces its best geometric guess, which in practice (5-8 out of 12 meaningful in current tests) is better than holding.

7.2 Memory Efficiency

The full SAGE system at 240MB is 14 times smaller than Mistral 7B at approximately 4GB when quantised for edge deployment. This footprint fits comfortably on NVIDIA Jetson Orin (16GB RAM, available at ~$500) confirming viability for real drone hardware without cloud dependency. The MultiCube architecture maintains this footprint regardless of total knowledge: adding a new domain adds one 16^3 cube (approximately 48MB) without modifying existing cubes.

7.3 Comparison to Existing Approaches

SAGE fills a gap that production systems do not address: updateable persistent memory that operates without connectivity. Skydio and DJI systems use pre-programmed rules and frozen neural networks — knowledge cannot be updated in the field. Hivemind uses swarm intelligence requiring inter-drone communication. RAG-augmented systems require LLM availability. SAGE provides the only architecture in which a drone can learn from a novel situation at step 10, have that learning persist to step 11, and use it without any external connection.

8. CONCLUSION

This paper has demonstrated graceful degradation in an LLM-augmented autonomous agent: zero defaults across 72 agent steps, including full SAGE-only operation after LLM shutdown, confirmed across two architectural variants. SAGE geometric retrieval is 3.6x faster than LLM inference. SAGEDivided working memory improves SAGE-only meaningful recall by 60% over MultiCube alone. Sequence encoding mechanisms are architecturally implemented and benign; their benefit requires higher memory density to measure.

Six gaps between this proof of concept and production deployment are identified and each has a concrete solution path: perception encoder for real sensor data, flight data collection for memory density, two-layer decision architecture for latency, sequence encoding evaluation after density increase, prompt engineering for LLM-SAGE integration, and safety certification for airspace deployment.

The architecture described here — geometric memory with structural anti-forgetting, continuous learning without retraining, and guaranteed fallback response — addresses a fundamental limitation of current drone AI systems. The geometry computes; the weights, and the connection, are not needed.

ACKNOWLEDGEMENTS

The author used Claude (Anthropic) as an AI research assistant throughout this work. All research decisions, results, and claims are the sole responsibility of the author.

REFERENCES

[1] Ahn, M., et al. (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. arXiv:2204.01691.

[2] Brohan, A., et al. (2023). RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818.

[3] Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing Machines. arXiv:1410.5401.

[4] Graves, A., et al. (2016). Hybrid computing using a neural network with dynamic external memory. Nature, 538(7626), 471-476.

[5] Kirkpatrick, J., et al. (2017). Overcoming catastrophic forgetting in neural networks. PNAS, 114(13), 3521-3526.

[6] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[7] Likov, I. (2026a). Memory for All: SAGE — Spatial Associative Geometric Embeddings. Zenodo. doi:10.5281/zenodo.19192937.

[8] Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. ICML 2021.

[9] Rana, K., et al. (2023). SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning. arXiv:2307.06135.